LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Modul 6: Protokol Komunikasi Telemetri Industri (MQTT)

Disusun oleh :

Nama : syafa sartika putri aryanti

NPM : 25782056

Kelas : TRI-3B

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra

TEKNOLOGI REKAYASA INTERNET

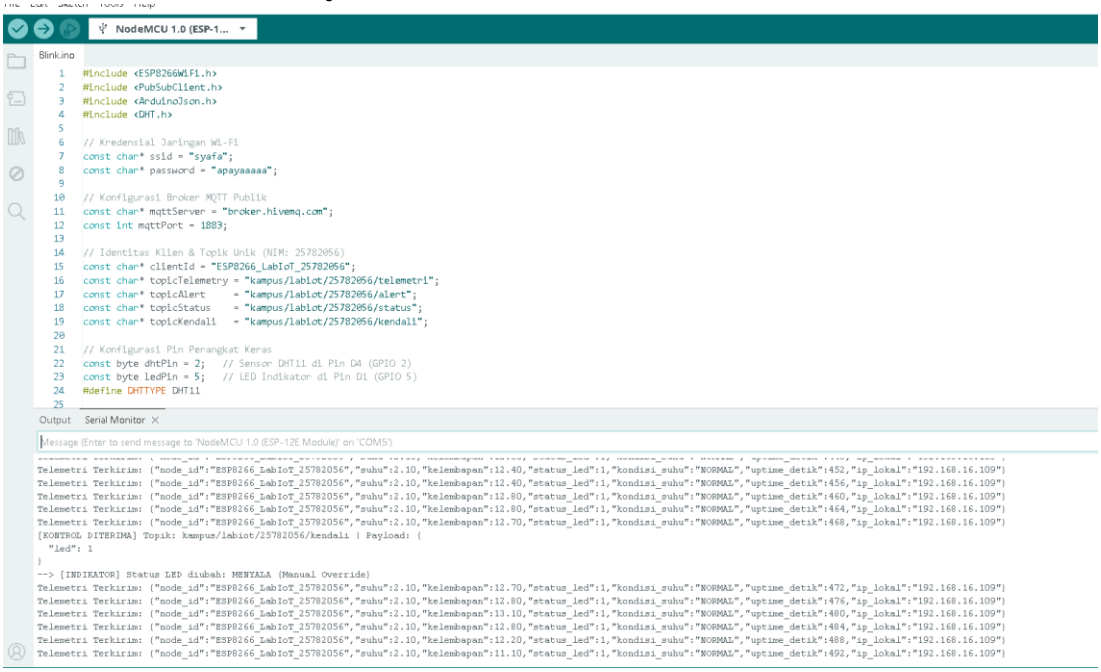
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

Hasil modifikasinya :



Kode modifikasinya :

Blink.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "syafa";
const char* password = "apayaaaaa";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// Identitas Klien & Topik Unik (NIM: 25782056)
const char* clientId = "ESP8266_LabIoT_25782056";
const char* topicTelemetry = "kampus/labiot/25782056/telemetri";
const char* topicAlert     = "kampus/labiot/25782056/alert";
const char* topicStatus    = "kampus/labiot/25782056/status";
const char* topicKendali   = "kampus/labiot/25782056/kendali";

// Konfigurasi Pin Perangkat Keras
const byte dhtPin = 2;   // Sensor DHT11 di Pin D4 (GPIO 2)
const byte ledPin = 5;   // LED Indikator di Pin D1 (GPIO 5)
#define DHTTYPE DHT11

```

Blink.ino

```
DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Interval pengiriman 4 detik

bool ledState = LOW;
bool alertSent = false; // Latch flag untuk mencegah pengiriman alert berulang

// Fungsi Callback saat menerima pesan dari broker (Kendali Manual)
void callback(char* topic, byte* payload, unsigned int length) {
  String message = "";
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }

  Serial.print("[KONTROL DITERIMA] Topik: ");
  Serial.print(topic);
  Serial.print(" | Payload: ");
  Serial.println(message);

  StaticJsonDocument<200> docIn;
  DeserializationError error = deserializeJson(docIn, message);
  if (!error) {
    if (docIn.containsKey("led")) {
      int cmdLed = docIn["led"];
      if (cmdLed == 1) {
        ledState = HIGH;
        digitalWrite(ledPin, ledState);
        Serial.println("--> [INDIKATOR] Status LED diubah: MENYALA (Manual Override)");
      } else if (cmdLed == 0) {
        ledState = LOW;
        digitalWrite(ledPin, ledState);
        Serial.println("--> [INDIKATOR] Status LED diubah: MATI (Manual Override)");
      }
    }
  }
}

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
```

Blink.ino

```
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("Alamat IP Lokal: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Menghubungkan ke MQTT Broker...");

    // Konfigurasi Last Will and Testament (LWT)
    const char* lwtTopic = topicStatus;
    int lwtQos = 1;
    bool lwtRetain = true;
    const char* lwtMessage = "{\"status\":\"OFFLINE\",\"keterangan\":\"Perangkat Terputus Tak Terduga\"}";

    if (client.connect(clientId, nullptr, nullptr, lwtTopic, lwtQos, lwtRetain, lwtMessage)) {
      Serial.println(" Sukses Terhubung!");

      // Publish status ONLINE setelah terhubung (Retain = true)
      StaticJsonDocument<128> docStatus;
      docStatus["status"] = "ONLINE";
      docStatus["keterangan"] = "Perangkat Beroperasi Normal";
      char statusBuffer[128];
      serializeJson(docStatus, statusBuffer);
      client.publish(topicStatus, statusBuffer, true);

      // Subscribe kembali ke topik kendali
      client.subscribe(topicKendali);
      Serial.print("Subskripsi Topik Kendali: ");
      Serial.println(topicKendali);

    } else {
      Serial.print(" Gagal, rc=");
      Serial.print(client.state());
      Serial.println(". Coba lagi dalam 5 detik...");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, ledState);

  dht.begin();
  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
}
```

```
void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Menjaga koneksi MQTT dan menangani pesan masuk secara asinkron

  unsigned long now = millis();
  if (now - lastTelemetryTime >= telemetryInterval) {
    lastTelemetryTime = now;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (!isnan(t) && !isnan(h)) {
      // 1. Klasifikasi Kondisi Suhu
      String kondisiSuhu = (t > 31.0) ? "PANAS" : "NORMAL";

      // 2. Penyusunan dan Pengiriman Paket Telemetri Rutin
      StaticJsonDocument<256> docOut;
      docOut["node_id"] = clientId;
      docOut["suhu"] = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);
      docOut["kondisi_suhu"] = kondisiSuhu;
      docOut["uptime_detik"] = millis() / 1000;
      docOut["ip_lokal"] = WiFi.localIP().toString();

      char buffer[256];
      serializeJson(docOut, buffer);
      client.publish(topicTelemetry, buffer);

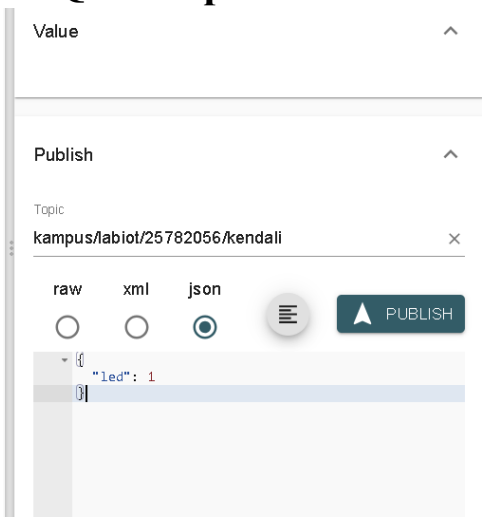
      Serial.print("Telemetri Terkirim: ");
      Serial.println(buffer);

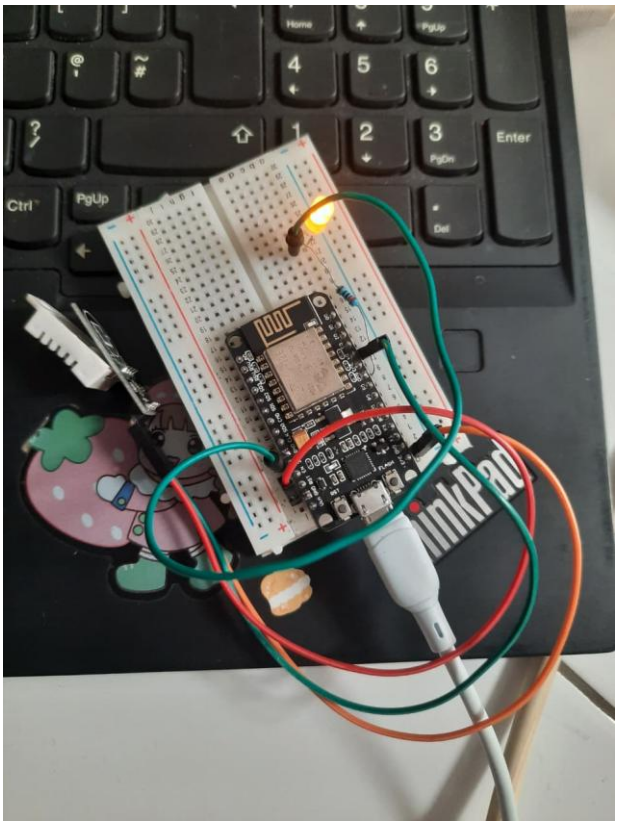
      // 3. Logika Edge Safety Engine (Ambang Batas > 31.0°C)
      if (t > 31.0) {
        // Otomatis nyalakan LED pengaman tanpa menunggu perintah luar
        ledState = HIGH;
        digitalWrite(ledPin, ledState);

        // Kirim alert khusus (menggunakan latch agar tidak spam setiap 4 detik)
        if (!alertSent) {
          StaticJsonDocument<200> docAlert;
          docAlert["status"] = "BAHAYA";
          docAlert["parameter"] = "OVERHEAT";
          docAlert["nilai_suhu"] = serialized(String(t, 2));

          char alertBuffer[200];
          serializeJson(docAlert, alertBuffer);
```

MQTT Explorer:





Penjelasannya :

Penjelasan Sistem Tugas 6.7: *Closed-Loop Industrial Safety & Automated Actuation*
Sistem ini dirancang untuk mensimulasikan lingkungan pengawasan industri otomatis berbasis IoT, di mana mikrokontroler (NodeMCU ESP8266) tidak hanya bertindak sebagai pengirim

data (*publisher*), tetapi juga mengambil keputusan lokal secara mandiri (*edge intelligence*) serta tetap mengakomodasi intervensi dari operator luar.

1. Akuisisi Sensor & Klasifikasi Kondisi Lingkungan
    - Cara Kerja: Sensor DHT11 secara berkala membaca parameter suhu dan kelembapan fisik dari lingkungan sekitar.
    - Implementasi: Di dalam program, nilai suhu dievaluasi menggunakan logika *edge computing*. Jika suhu $\leq 31.0°C$, sistem mengklasifikasikannya sebagai "NORMAL". Sebaliknya, jika suhu $> 31.0°C$, status otomatis berubah menjadi "PANAS" dan disisipkan ke dalam format payload JSON telemetri rutin.
2. Kanal Peringatan Bahaya Mandiri (*Dedicated Alert Topic & State Latch*)
    - Logika Keselamatan (*Edge Safety Engine*): Ketika suhu terdeteksi melampaui ambang batas kritis $> 31.0°C$, sistem langsung mengambil tindakan pengamanan secara mandiri dengan menyalakan LED indikator (pin D1) tanpa harus menunggu perintah dari server atau operator.
    - Mekanisme *State Latch* (alertSent): Agar *broker* MQTT tidak mengalami *spam* data peringatan setiap 4 detik, program menggunakan variabel penanda (*flag*). Pesan peringatan darurat ke topik .../alert dengan format JSON (BAHAYA, OVERHEAT) hanya dikirimkan sekali pada saat kondisi bahaya pertama kali terdeteksi, dan akan mereset statusnya kembali setelah suhu turun ke ambang normal.
3. Mekanisme *Last Will and Testament* (LWT)
    - Fungsi Keandalan: LWT adalah fitur bawaan protokol MQTT untuk menangani kegagalan koneksi tak terduga (*unexpected disconnection*).
    - Implementasi: Saat pertama kali terhubung ke *broker*, perangkat mendaftarkan pesan wasiat pada topik .../status dengan status awal ONLINE. Jika perangkat mendadak terputus (misalnya kabel daya USB dicabut secara sengaja), *broker* MQTT akan secara otomatis mempublikasikan pesan wasiat berstatus OFFLINE (Perangkat Terputus Tak Terduga) kepada seluruh sistem pemantau.
4. Kendali *Manual Override* Jarak Jauh
    - Fleksibilitas Operasional: Meskipun sistem berjalan otomatis berdasarkan ambang batas suhu, operator di luar tetap memiliki hak alih kendali (*manual override*) saat kondisi lingkungan normal.
    - Alur Eksekusi: Melalui aplikasi MQTT Explorer, operator mengirimkan perintah berformat JSON ({"led": 1} atau {"led": 0}) ke topik .../kendali. Pesan ini diterima secara asinkron oleh fungsi *callback* di mikrokontroler, yang kemudian langsung mengeksekusi perubahan status nyala/mati pada LED indikator.